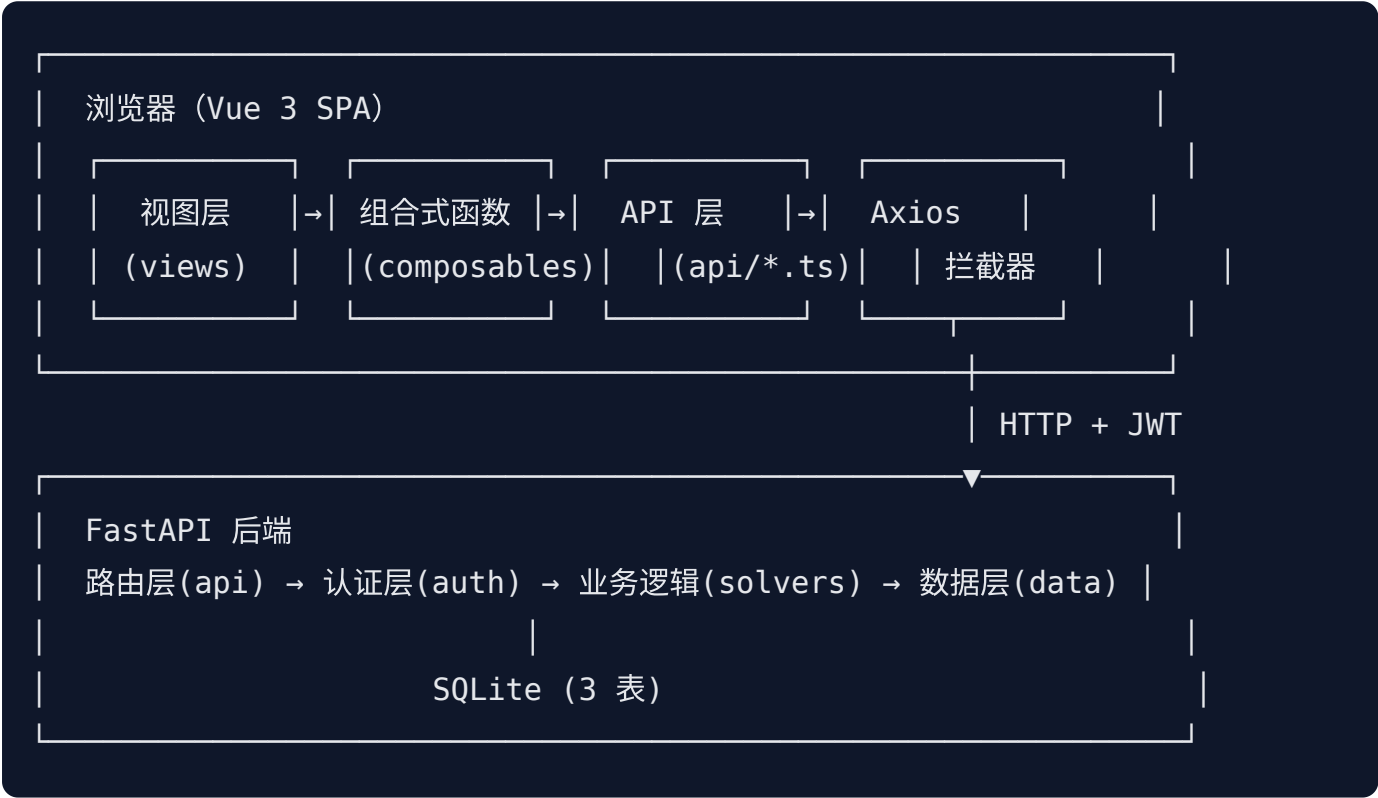


技术架构

对应任务书：九.3 开发工具与运行环境 / 九.6 程序计算流程。
本平台为前后端分离 Web 应用：**Vue 3 + TypeScript** 前端 + **FastAPI + Python** 后端 + **SQLite** 存储，支持 Docker 与 Windows 便携版两种部署。

总体架构



一、后端架构（FastAPI）

采用分层架构，职责清晰、便于测试：



1. 路由层 app/api/

文件	端点	说明
----	----	----

health.py	GET /api/health	健康检查（前端顶栏 LED 轮询）
projects.py	项目 CRUD + POST /api/projects/{id}/calculate	计算：派生输入 → 调求解器 → 写回结果
snapshots.py	快照归档 / 恢复 / fork / 删除	
auth/router.py	注册 / 登录 / 获取当前用户 / 删除账户	

- main.py 挂载路由、配置 CORS、启动时建表、SPA 静态文件回落（便携版模式）。
- 参数校验由 Pydantic 模型自动完成；超筋时求解器抛 ValueError，/calculate 捕获并返回 400。

2. 认证层 app/auth/

- dependencies.py::get_current_user：所有需隔离端点的统一入口，声明 Depends(get_current_user) 即注入当前用户并自带按 user_id 隔离。
- repository.py：用户数据访问（注册、按用户名查询）。
- 密码 bcrypt 哈希；JWT（HS256）7 天有效期，无状态、不依赖 DB。

3. 业务逻辑层 app/solvers/（计算引擎，核心）

```
solvers/
├─ common.py          # 共享：h0/αs/ξ/As、抗剪、等跨连续梁系数表（2~5跨）、配筋候选
├─ derive.py          # 扁平参数 → 各构件输入（荷载、跨度、截面派生）
├─ materials.py       # 材料强度与分项系数（单一来源）
├─ slab/              # 板：solver.py（编排）+ utils.py（按间距选筋）
├─ beam/              # 次梁：T形/矩形正截面、支座边缘调整
└─ main_beam/         # 主梁：三等分集中力包络（表14-4，3跨）+ 吊筋
```

- 每个构件独立子包，共享公式提取至 common.py；每个阶段输出均为 Pydantic 模型，可独立测试。
- 计算流程见 [开发指南 § 计算流程](#)，公式见 [计算方法与公式](#)。

4. 数据访问层 app/data/

- `connection.py`：SQLite 连接管理 + 建表 DDL（核心模块，非占位，被 `auth / data repo + main lifespan` 引用）。
- `project_repository.py`：项目 / 快照数据访问，参数化查询防注入。

数据库表结构（SQLite 三表）

表	关键字段	说明
users	id（主键）、username（唯一）、password_hash	注册用户； 无 AUTOINCREMENT （避免删用户后 id 计数器误导）
projects	id、user_id（外键 ON DELETE CASCADE）、name、data（JSON blob）、updated_at	每用户多项目； <code>data</code> 存全部参数与计算结果
snapshots	id、project_id（外键 CASCADE）、name、data（JSON）、created_at	项目历史快照

二、前端架构（Vue 3）

采用 **Composition API + 模块级单例** 状态管理（不引入 Pinia/Vuex）。

1. 分层

层	目录	职责
视图层	<code>views/</code>	9 个页面（见 功能详解 ）
组件	<code>components/layout</code> 、 <code>common</code> 、 <code>diagrams</code> 、 <code>report</code>	外壳、通用状态、计算简图、计算书渲染
组合式函数	<code>composables/</code>	模块级 <code>ref</code> 实现全局状态单例
API 层	<code>api/</code>	Axios 封装，统一错误拦截
路由	<code>router/</code>	懒加载 + 导航守卫（未登录跳登录）

2. 关键组合式函数

Composable	职责
useAuth	登录 / 注册 / 登出、JWT 持久化、用户偏好加载
useProject	当前项目数据、800 ms 防抖自动保存、确认计算、快照操作
useTheme	暗亮暖三态主题，按用户分键持久化
useSidebar	侧栏折叠 + 移动端抽屉（matchMedia）
useHealth	后端健康轮询
useReportDocument	纯函数 buildReportDoc(data) 装配计算书章节树
useResistingMoment	纯函数 computeMu 计算抵抗弯矩
useBeamLayout	纯函数 梁跨数 / 跨度 → SVG 坐标（含五跨等效）

纯函数组合式（装配器、Mu、布局）均带单测，与渲染组件解耦，保证可测试性。

3. 数据形状

```
data = {
  structure: { L1, L2, slab_thickness, beam_*, main_beam_*,
column_width,
              slab_spans, beam_spans, main_beam_spans,
*_stirrup_diameter },
  loads:      { reinforced_concrete_weight, terrazzo_surface,
               plaster_thickness, plaster_weight, live_load },
  report:     { college, major, name, student_id, advisor, date }, // 封面
  slab:       { result, initialized },
  beam:       { result, initialized },
  main_beam:  { result, initialized },
}
```

- 各构件不再单独存 input，/calculate 时由 solvers/derive.py 从 structure + loads 派生。
- 材料值与分项系数固定在后端 materials.py，前端只读展示。

三、鉴权与用户隔离

- **注册不自动登录**：建账户后切回登录页（保留用户名、清密码）。
 - **登录失败统一 401**「用户名或密码错误」（防枚举爆破攻击或ddos攻击）。
 - **JWT 7 天无状态**：过期后下次 API 调用 → 401 → 前端 Axios 拦截器清登录态跳 `/login`。
 - **数据隔离**：项目 / 快照按 `user_id` 存取（`get_current_user` 注入），用户间互不可见。
 - **偏好隔离**：主题、侧栏按用户分 `localStorage` 键（`ccd-theme:<uid>`、`ccd-sidebar:<uid>`），登录载入、登出回退全局、注销时清除。
-

四、三主题令牌系统

视觉上保留 **Vue + Element Plus 不切栈**，通过设计令牌 + 克制配色 + 三主题实现。

- **单一令牌表** `frontend/src/assets/styles/tokens.css`：三个 `[data-theme="dark|light|warm"]` 语义令牌块 + 一个 `:root` Element Plus 桥接。
 - **EP 桥接**：把 `--el-*` 变量映射到语义令牌；色阶用 `color-mix(in srgb, 基色 N%, var(--card))` 派生——light/warm 主题 `--card` 为白精确复现 EP 算法，dark 主题混入深色 card 得暗色 tint。一套定义自动适配三主题。
 - **主题切换走 `data-theme` 属性**（非 EP 的 `html.dark`）；`main.ts` 挂载前预设防闪烁；默认 `light`。
 - **中性令牌**：间距 `--space-1..10`（4 px 网格）、字号、半径、动效、z-index、布局尺寸；全局工具类 `.block/.muted/.guard`。
-

五、外壳与响应式

- `AppLayout`（flex 列：TopBar + [Sidebar | main]）、`AppHeader`（CSS 手绘健康度 LED + 折叠钮 + UserDropdown）、`AppSidebar`（`<router-link>` 导航，激活态 = 主色左边框）。
 - **响应式**：`<900 px` 侧栏变 absolute overlay + 半透明遮罩，路由跳转后自动收起；桌面可折叠（图标态 56 px / 展开态 208 px）。
-

六、构建与部署形态

形态	入口	前端托管	适用
开发	<code>uvicorn --reload + vite dev</code>	Vite dev server（代理 /api）	开发
Docker	<code>docker compose up</code>	Nginx 容器（反代 /api → backend:8000）	Linux 部署
Windows 便携版	<code>ConcreteCourseDesign.exe</code>	后端 <code>SERVE_STATIC=1</code> 直接托管 dist	单机演示

详见 [安装与运行](#)。